

Zero-Sum Deletion Certificates versus Intrinsic Support in Quantum Compilation

Sze Chun Yiu
sze-chun.yiu@fysik.su.se

25 August 2026

Abstract

A finite-support certificate can be exact for its proof language and still be loose as a description of the compiler it certifies. We formalize this separation with an alphabet-restricted zero-sum invariant. For a finite abelian signature group H and allowed alphabet $A \subseteq H$, let $\text{zsf}(H; A)$ be the largest length of a zero-sum-free word over A . In the deletion language whose only shortening step removes a nonempty proper zero-sum subword from a nonzero-total word, the exact uniform terminal complexity is $\text{zsf}(H; A)$. Every longer word is reducible, while a longest zero-sum-free word is a matching terminal witness. A production compiler inherits the matching lower bound only if that witness is realizable and no additional rule can reduce it.

Two quantum-compilation families give opposite controls. In a one-Tag, three-block compiler, the production alphabet realizes $\mathbb{F}_2^2$; the deletion certificate and independently proved intrinsic support are both two. In a two-block dependent-triple compiler with a shared two-bit Tag, the production alphabets realize basis obstructions in $\mathbb{F}_2^5$, so the rank-only certificate is exactly five, whereas a whole-system Tag-relocation theorem gives intrinsic support exactly one. For a disjoint product of t components, the corresponding budgets are $5t$ and t . A direct support enumerator therefore has search-volume ratio $\Theta(n^{4t})$ under the stated fixed-budget model. This is not an unrestricted proof or algorithm lower bound. It shows that a support number belongs to a compiler only after an intrinsic lower witness; otherwise it belongs to a named normalization or certificate language.

Keywords: quantum compilation; certificate complexity; support normalization; zero-sum deletion; exact optimization

1. Introduction

Support bounds turn unbounded exact compiler searches into finite ones. Their numerical value can nevertheless answer three different questions:

1. What support does the compiler intrinsically require?
2. What support can a particular normalization attain?
3. What support can a restricted proof language certify?

These quantities coincide in some families and differ sharply in others. The alphabet-Davenport normal-form theorem supplies a general upper certificate. Here we identify the exact complexity of its deletion language and compare it with independently established compiler support.

The distinction is operational. If a search implementation enumerates all supports up to a certificate ceiling, a loose proof language can impose a large, avoidable search volume even when the compiler

has a much smaller normal form. Conversely, a small normal form is not intrinsic until a lower witness rules out smaller support.

Our contributions are:

1. an exact terminal-complexity theorem for zero-sum deletion;
2. a realization criterion separating abstract terminal words from production compiler states;
3. a tight one-Tag, three-block control with certificate and intrinsic support two;
4. a strict dependent-triple separation between exact certificate complexity five and intrinsic support one; and
5. a disjoint-product construction with additive gap $4t$ and a precisely scoped direct-enumeration consequence.

2. Three support quantities

Fix a compiler family $\mathcal{F}$ and objective C .

2.1 Intrinsic support

The intrinsic support $\kappa(\mathcal{F}, C)$ is the least k such that every instance has an exact optimum of support at most k , together with an independent witness showing that support $k - 1$ cannot always suffice.

2.2 Normalization ceiling

A transformation N may show that every feasible configuration has a no-more-expensive representative of support at most B_N . Without a matching compiler lower witness, B_N belongs to the triple $(\mathcal{F}, C, N)$, not intrinsically to $(\mathcal{F}, C)$.

2.3 Certificate complexity

A proof language P restricts visible information and legal inference. Write $\beta_P(\mathcal{F}, C)$ for the least uniform support budget that P can certify on its production scope. Soundness gives

$$\kappa(\mathcal{F}, C) \leq \beta_P(\mathcal{F}, C),$$

but equality requires additional mathematics.

Quantity	Owned by	Lower-witness requirement
Intrinsic support κ	compiler family and objective	compiler obstruction
Normalization ceiling B_N	compiler, objective, and transformation	only for intrinsic interpretation
Certificate complexity β_P	compiler scope and proof language	terminal witness realizable in production

3. Exact zero-sum deletion complexity

Let H be a finite abelian group and $A \subseteq H$. A certificate word $v_1 \cdots v_w$ has nonzero total. The only legal shortening removes a nonempty proper subword whose sum is zero. Define

$$\text{zsf}(H; A) = \max\{|W| : W \text{ is zero-sum-free over } A\}.$$

Theorem 1 (exact abstract certificate complexity). The maximum terminal length among all nonzero-total words over A is exactly $\text{zsf}(H; A)$.

Proof. A word longer than $\text{zsf}(H; A)$ contains a nonempty zero-sum subword. Its nonzero total prevents that subword from being the whole word, so a legal deletion exists. Conversely, a longest zero-sum-free word has nonzero total and admits no legal deletion. $\square$

Corollary 2 (production realization). If every production word lies over A , its certificate ceiling is at most $\text{zsf}(H; A)$. The ceiling is exact only if production realizes a longest zero-sum-free word and the certificate language has no other rule that reduces that state.

For $H = \mathbb{F}_2^d$, $\text{zsf}(H; A) \leq d$; an alphabet containing a basis has equality.

4. Tight control: one-Tag, three-block compiler

A frame coordinate in the one-Tag, three-block compiler carries partner and Tag bits. Denote this family by $\mathcal{C}_{1\text{T}3\text{B}}$. Its production alphabet realizes a basis of $\mathbb{F}_2^2$, giving

$$\beta_{\text{del}}(\mathcal{C}_{1\text{T}3\text{B}}) = 2.$$

The all-size zero-sum exchange proves support at most two. A complete exact $n = 2$ production instance supplies the lower witness: its unrestricted optimum has cost 5 and support two, whereas exhaustive minimization over all 13,824 feasible support-at-most-one configurations has cost 6. Therefore

$$\kappa(\mathcal{C}_{1\text{T}3\text{B}}) = 2 = \beta_{\text{del}}(\mathcal{C}_{1\text{T}3\text{B}}).$$

This control shows that the certificate is not inherently loose. Tightness depends on whether its terminal obstruction is also an intrinsic compiler obstruction.

5. Strict separation: dependent-triple compiler

The second family, denoted $\mathcal{C}_{\text{DT}}$, has two rank-two dependent-triple blocks with a shared two-bit Tag under the unit structural objective. Each block has generators R_0, R_1, R_2 with $R_2 = R_0 R_1$. The partner, block, and Tag syndromes form a five-bit quotient. Direct evaluation maps the five realized Pauli words (XIIII, IXIXI, IYIYI, IIXIXI, IYIYI) and, for the second block, (YIIII, IXIII, IYIII, IIXIII, IYIII) to basis words of the quotient. Thus the production block-deletion alphabets span the quotient and realize basis words. Theorem 1 and production realization therefore give

$$\beta_{\text{del}}(\mathcal{C}_{\text{DT}}) = 5.$$

A stronger transformation goes beyond rank-only deletion. First, the support-two parent theorem localizes each dependent triple to an anticommuting core. At every non-core column, deleting the redundant frame letters refunds at least four units. If the two blocks choose distinct cores, the previous Tag has cost at least four and the new two-core Tag costs eight, so the combined credit pays the reconstruction. If they choose the same core, the new Tag costs four and the remaining alignment penalty is at most three. These exhaustive local inequalities are independent of the number of untouched columns and therefore compose to all sizes. The resulting whole-system Tag relocation leaves each frame generator at support at most one without increasing cost. Support zero is infeasible because an identity frame cannot anticommute with its partner. Hence

$$\kappa(\mathcal{C}_{\text{DT}}) = 1, \quad \beta_{\text{del}}(\mathcal{C}_{\text{DT}}) - \kappa(\mathcal{C}_{\text{DT}}) = 4.$$

There is no contradiction: a basis word blocks zero-XOR deletion while the successful proof changes auxiliary Tag structure that the rank-only language holds fixed.

6. Disjoint product amplification

Let $\mathcal{C}_{\text{DT}}^{\times t}$ be the disjoint product of t components on disjoint coordinates, with additive support and no cross-component move.

Theorem 3. For every $t \geq 1$,

$$\beta_{\text{del}}(\mathcal{C}_{\text{DT}}^{\times t}) = 5t, \quad \kappa(\mathcal{C}_{\text{DT}}^{\times t}) = t.$$

Proof. Componentwise upper bounds add. A realized basis obstruction in each component gives a certificate lower bound of five per component. Support zero is infeasible in every component, and the support-one normalization acts independently. Thus both lower and upper intrinsic bounds equal one per component. $\square$

The additive gap is $4t$. This construction amplifies one mechanism; it is not evidence for a second, independent source of separation.

6.1 Consequence for direct support enumeration

For fixed local alphabet and fixed budget B , an enumerator that explicitly visits all coordinate supports of size at most B has leading growth $\Theta(n^B)$. Under that declared model, the disjoint product yields

$$\Theta(n^{5t}) \quad \text{versus} \quad \Theta(n^t),$$

and therefore ratio $\Theta(n^{4t})$. The statement is not a complexity-class lower bound and does not constrain algorithms that avoid direct support enumeration.

7. Relation to prior work

Sparse integer optimization already studies support bounds and lower constructions [1]. Classical and modern zero-sum theory owns Davenport constants, restricted alphabets, weighted variants, basis obstructions, and direct sums [2,3]. Proof-complexity and formal-methods research already distinguishes object difficulty from lower bounds internal to a proof language.

The residual contribution is the exact production separation in quantum compilation. The same algebraic certificate is tight for the one-Tag, three-block compiler, loose by a factor of five for the dependent-triple compiler, and additively unbounded under disjoint products. The latter normalization also identifies the missing proof operation—whole-system Tag reconstruction - instead of merely reporting a smaller number.

8. Reproducibility and limitations

Finite-group controls, binary basis obstructions, production alphabets, and product formulas have been checked with an independent deterministic verifier. Theorems 1 and 3 supply all-size authority.

The exact abstract theorem concerns only the stated deletion language. A production lower bound requires realization of its terminal witness. We prove no lower bound for every local, syndrome-preserving, or unrestricted proof system. The disjoint product forbids cross-component transformations by definition. The enumeration ratio belongs only to the direct support model. Finally, structural support does not imply a hardware speedup or any physical quantum-resource advantage.

9. Conclusion

A certificate can be sound, useful, and internally exact while measuring the wrong layer for an intrinsic interpretation. The alphabet-restricted invariant identifies the expressive ceiling of zero-sum deletion. The one-Tag, three-block compiler shows that this ceiling can coincide with intrinsic support. The dependent-triple compiler shows that a whole-system transformation can cross it sharply.

The reporting rule is therefore simple: call a support number intrinsic only after an independent compiler lower witness. Otherwise identify the normalization or proof language that owns it.

Data and code availability

The arXiv ancillary package contains the production-alphabet records, exact one-Tag, three-block lower witness, dependent-triple support-one normalization checks, and product-formula verifier. The finite enumerations corroborate the displayed construction and local inequalities; Theorems 1 and 3 carry the abstract and product claims. A permanent archival identifier will replace the ancillary-package reference in the journal version.

References

1. I. Aliev, J. A. De Loera, F. Eisenbrand, T. Oertel, and R. Weismantel, “The Support of Integer Optimal Solutions,” *SIAM Journal on Optimization* **28**, 2152-2157 (2018). DOI:

10.1137/17M1162792

2. G. Wang, “The universal zero-sum invariant and weighted zero-sum for infinite abelian groups,” *Communications in Algebra* **53** (2025). DOI: 10.1080/00927872.2024.2418017
3. M. Freeze and W. A. Schmid, “Remarks on a generalization of the Davenport constant,” *Discrete Mathematics* **310**, 3373-3389 (2010). DOI: 10.1016/j.disc.2010.07.028